

CSE321 Theory Assignment 02

Unix/Linux systems use the **rwX** (owner–group–others) permission model to control access to files. Although simple, it becomes difficult when different users need different permissions on the same file. **Extended ACLs** solve this by allowing very structured permissions for specific users and groups beyond the basic three categories. This report explores the shift from traditional Unix permissions to Extended Access Control Lists (ACLs), highlighting how ACLs overcome the limitations of the 3-tier model.

The traditional Unix permission model follows a **3-tier structure**: Owner (User), Group, and Others which presents significant bottlenecks in modern, complex multi-user environments.

Standard Permission Bottlenecks:

The primary problem is the lack of granularity; as it is not possible to set standard permissions on a case by case basis as per individual users or groups. When a third user needs access privileges that are not similar to the category of others, the administrator is forced to either change the ownership of the group or add that user to the existing group, an action that can easily enable them access other files owned by the group. In the normal Unix model, one cannot assign different permissions to different users.

Under the standard Unix model, different permissions cannot be configured for different individual users. Let us consider we have a sensitive file which is “Budget”

- **The Owner (Manager):** Needs full **rwX** access.
- **The Accountant:** Needs **rw-** (read/write) access.
- **The Intern:** Needs **r--** (read-only) access.

The Isolation Failure: In a standard 3-tier system (Owner, Group, Others), the Manager is the owner and the Accountant can be placed in the file's group with write permissions. However, the Intern cannot do it. To give the Intern read access, the Manager would have to grant "read" permissions to the "Others" category. This results in a security failure because "Others" applies to everyone, every single user on the system can now read the sensitive budget file just to allow the Intern to see it.

Mechanism of Extended ACLs:

ACLs provide a more flexible system that works alongside traditional Unix permissions to manage access

POSIX ACL Entry Structure and Metadata

A POSIX ACL entry extends the standard permission model by allowing multiple entries for different users and groups. These entries are stored as extended attributes within the file system. Each rule typically follows a specific format:

- a) u:uid:perms:** Sets permissions for a specific user (by name or UID).
- b) g:gid:perms:** Sets permissions for a specific group (by name or GID).
- c) m:perms:** The effective rights mask, which limits the maximum permissions possible for all users and groups (except the owner and "others")
- d) o:perms:** Permissions for users not in the file's user group.

When ACLs are active, the standard `ls -l` output displays a "+" sign indicating that extended permissions are present and must be viewed with specialized tools.

CLI Implementation: chmod vs. setfacl/getfacl

Managing permissions requires different tools depending on whether one is using standard or extended models:

- **Standard (chmod):** Primarily used to change the 3-tier mode (Owner, Group, Others).
- **Extended (setfacl and getfacl):**
 - a) setfacl:** Used to modify (-m), remove (-x), or strip all (-b) ACL entries.
 - b) getfacl:** Used to retrieve and display the full list of permissions, including the owner, group, standard permissions, and any extended user/group entries.

Priority Logic and Permission Check

When the OS checks access for a user, it follows a specific logic to determine the effective rights:

1. **Specific User Entry:** If an ACL entry exists for the specific user, those permissions are applied.
2. **Group and Mask Logic:** If no specific user entry exists, the OS checks relevant group entries. However, these are filtered by the effective rights mask. The mask acts as a "ceiling"; even if a group entry says rwx, if the mask is r--, the effective permission is only read.
3. **Default ACLs:** For directories, Default ACLs can be set. While they do not change the directory's own permissions, they ensure that any new files created within that directory automatically ensure the specified ACL entries